# 데이터베이스

D = < S, O, C >

## Relation – 제약 조건 (Constraint)

오병우

컴퓨터공학부

kit 국립금오공과대학교
Kumoh National Institute of Technology

# 개요

version

Configuration Management (형상 관리)

- 무결성 (Integrity, 완전한 상태) 규칙 (rule)
  - 실세계에서의 특정 제약 조건을 DBMS에 알려서 불가능한 값 형상(configuration)이 발생하지 않도록 방지
  - (ex) 사용자 정의 무결성
    - 공급자 번호는 Snnnn 형태
    - 부품 번호는 Pnnnn 형태
    - 공급자 상태 값은 반드시 1~100 범위내
    - 부품 중량은 0보다 큼
    - 배송 수량은 100의 배수
- 모든 데이터베이스에 적용되는 두 가지 일반적인 무결성 규칙
  - 개체 무결성 (Entity integrity rule): primary key
  - 참조 무결성 (Referential integrity rule): foreign key

# Key

attribute의 집합

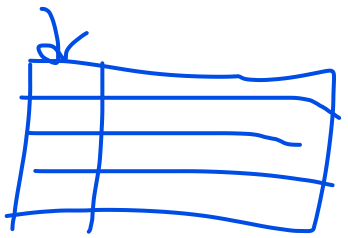
- 일반적인 의미
  - 자물쇠를 푸는 열쇠
  - 중요하고 핵심적인 부분 (key man)
- Relation에서의 의미
  - 투플을 유일하게 식별할 수 있는 애트리뷰트 집합
  - 애트리뷰트 전부를 사용해도 식별 가능하기는 함
  - 일반적으로는 애트리뷰트의 일부를 사용함
- 흔히 혼용해서 사용
  - Primary Key 또는 Candidate Key로 혼용하므로 문맥의 의미를 파악하고 유의해야 함

# 후보키 (Candidate key)

이론적으로는 Primary key보다 중요

- 후보키
  - 무엇에 대한 후보? 정답) Primary Key
  - Relation에서 tuple을 유일하게 구별할 수 있는 attribute set

시험에서 Primary key라고 써서 틀리는 경우 많음



- R의 애트리뷰트 K (복합 애트리뷰트 가능) : candidate key
  - 릴레이션 R(A1, A2, ..., An)에 대한 애트리뷰트 집합 K = { Ai , Aj , ..., Ak }로서 아래 성질을 만족하면 후보키
- 다음과 같은 두 가지 (시간 독립적) 속성을 충족:
  - uniqueness (유일성): 학생 릴레이션의 (학번, 학과)
    - 주어진 시간에 R의 두 tuple은 K에 대해 동일한 값을 갖지 않음
  - minimality (최소성): 학생 릴레이션의 (학번)
    - K가 복합 애트리뷰트면, 유일성을 파괴하지 않고는 K의 어떤 구성 요소도 제거 불가능
    - 필요없는 attribute가 있으면 candidate key가 아님 (super key라고 부름)
- 모든 relation에는 적어도 하나의 후보 키 존재 (∵중복 튜플 없음, 모든 애트리뷰트 집합)

왜냐하면

# 프라이머리키 (Primary key)

- 프라이머리키 (주 키, 기본 키)
  - 후보 키들 (candidate keys) 중에서 데이터베이스 설계자가 지정한 하나의 키
    - 모든 relation에는 프라이머리키 존재
  - Tuple-레벨 주소 지정 (addressing) 메커니즘
    - 내용에 의한 참조 (Content Reference, Associative Addressing)

주소에 의한 참조가 아님

- 질의 결과 예제
  - (ex) P(P#, PNAME, COLOR, WEIGHT, CITY)

P 릴레이션에서 P#가 Primary Key일 경우

```
SELECT *
FROM   P
WHERE P# = 'P3';
```

(많아야) 한 개의 tuple

```
SELECT *
FROM   P
WHERE CITY = 'Paris';
```

예측 불가능한 수의 tuple

여러 개 있을 수 있음

# 대체키 (Alternate key)

- 대체키
  - 프라이머리키가 아닌 후보키 대체키(Alternate Key)는 후보키(Candidate Key) 중에서 기본키(Primary Key)로 선택되지 않은 나머지 키를 말해요.
  - 공급자: S(S#, SNAME, STATUS, CITY)
    - 1. candidate key: S#, SNAME
    - 2. primary key: S#
    - 3. alternate key: SNAME

Content Referencing

- 관계형 모델에는 “associative addressing (연관 주소 지정)” 필요
  - 주소가 아닌 “값”으로 tuple을 select 가능
  - 모든 관계형 주소 지정 방법은 명확하게 associative (연관적)
    - (i.e., 위치-기반이 아닌 값-기반)
  - 논리적 레벨에서 처리
    - 물리적 레벨에서 associative memory 같은 하드웨어가 필요한 것이 아님
- 정리하면: Candidate Key = Primary Key + Alternate Key

# 개체 무결성 규칙 (The Entity Integrity Rule)

- Primary key는 null 일 수 없음
  - Base (실제 저장된) relation의 primary key를 구성하는 애트리뷰트는 null을 허용하지 않음

- Null (존재하지 않음, 알 수 없음, 적용할 수 없음) (≠ blank or zero)

| Null Tag | Value |
|---|---|

  - 어떤 이유로 정보가 누락됨
    - e.g., 속성이 적용되지 않거나 값을 알 수 없음, etc.
  - 해당 속성의 실제 값을 나타내지 않는 것으로 관례상 이해되는 값 또는 표현
  - 정보 부재를 명시적으로 표현하는 특수한 데이터 값
    - ① 알려지지 않은 값 (unknown value)
    - ② 해당 없음 (inapplicable)
  - SQL에서 IS 또는 IS NOT으로 비교
    - = 또는 != 사용하지 않음

# 외래 키 (Foreign key)

- 외래 키 (반대 방향으로는 안됨)
  - 어떤 릴레이션 R1의 primary key 값과 일치해야 하는 값을 가진 릴레이션 R2의 속성(복합 가능)
    - R1과 R2가 반드시 다를 필요는 없음 (자기 자신 참조 가능)
  - R1: 참조되는 (referenced) relation (target relation)
  - R2: 참조하는 (referencing) relation
- 애트리뷰트 FK (복합 가능) of base relation R2: 외래 키 (foreign key)
  - 다음과 같은 두 가지 시간 독립적 속성 충족:
    - 1. FK의 각 값은 완전히 null이거나 완전히 null이 아님
    - 2. FK의 null이 아닌 값과 PK (primary key)의 값이 동일한 tuple이 R1에 존재해야 함
- Referential diagram

R1 for S# R2 R1 for P#

S(<u>S#</u>, SNAME, STATUS, CITY) ← SP(<u>S#, P#</u>, QTY) → P(<u>P#</u>, PNAME, COLOR, WEIGHT, CITY)

# 외래 키 (Foreign key)

- Foreign key 키 및 해당 primary key: 동일한 도메인
- Foreign key는 참조하는 릴레이션에서 primary key의 구성 요소일 필요는 없음
  - DEPT (DEPT#, …, BUDGET, …) : 여기에서는 primary key
  - EMP (EMP#, …, DEPT#, …, SALARY, …) : 여기에서 DEPT#는 primary key가 아님
- 한 relation은 참조되는 relation이면서 참조하는 relation일 수도 있음
  - Rn → R(n-1) → R(n-2) → … → R2 → R1 : 참조 경로 (referential path)
- Relations R1과 R2는 반드시 다를 필요는 없음 (자기 자신 참조 가능)
  - EMP(EMP#, …, SALARY, …, MGR_EMP#, …) : MGR_EMP#는 팀장
- 순환적 참조 가능
  - Rn → R(n-1) → R(n-2) → … → R2 → R1 → Rn (가능)
- Foreign keys는 null (not exist)일 수 있음
  - MGR_EMP#의 값 (사장은 null)
- Foreign-to-primary-key match
  - Tuple들간의 관계

외래키(Foreign Key)는 다른 테이블의 기본키(Primary Key)를 참조하는 키예요. 즉, 두 테이블을 연결해주는 "다리" 역할을 하는 속성이죠.

참조 관계: 한 테이블의 열(Column)이 다른 테이블의 Primary Key를 참조함

| 학번 (PK) | 이름 | 학과 |
|---|---|---|
| 2026001 | 김철수 | 컴퓨터공학 |
| 2026002 | 이영희 | 경영학 |

여기서 Enrollment 테이블의 학번(FK)은 Student 테이블의 학번(PK)을 참조해요

# 참조 무결성 규칙 (The Referential Integrity Rule)

- 데이터베이스에는 매칭되지 않는 외래 키 값이 없어야 함
  - B가 A를 참조하는 경우 A가 반드시 존재해야 함

S: Supplyer (공급자)
P: Part (부품)
SP: 어떤 공급자가 어떤 부품 공급하는지

- S, P, SP에서 S의 S# 변경 시에 SP를 어떻게 처리?
  - 특정 tuple의 S#를 S1에서 S6로 변경할 때 처리 방법
    - 허락하지 않음
    - S#에서 변경후 SP에서도 변경
    - S#에서 변경후 SP에서 대응값은 NULL로 처리

S(<u>S#</u>, SNAME, STATUS, CITY) ← SP(<u>S#, P#</u>, QTY) → P(<u>P#</u>, PNAME, COLOR, WEIGHT, CITY)

# 외래 키 (Foreign Key) 규칙

- 외래 키에 대한 질문
  - Foreign key에 null 넣을 수 있나요? → 가능
  - Foreign key 참조 대상 tuple 삭제
    1) RESTRICT: 참조하는 tuple이 없을 때만 삭제 가능 (foreign key로 참조되고 있다면 거부됨)
       - Oracle은 on delete를 생략하면 기본적으로 삭제 방지
    2) CASCADES: 참조하는 tuple도 연쇄적으로 따라가면서 삭제됨
       - (ex) R3 → R2 → R1
    3) SET NULL: 매칭되는 참조 tuple의 foreign key에 null 넣음
  - Foreign key 참조 대상인 primary key 값 갱신
    1) RESTRICT: 참조하는 tuple이 없을 때만 갱신 가능 (foreign key로 참조되고 있다면 거부됨)
    2) CASCADES: 참조하는 tuple도 연쇄적으로 따라가면서 갱신됨
    3) SET NULL: 매칭되는 참조 tuple의 foreign key에 null 넣음

# CASCADE 예제

- MySQL 5.5 또는 MariaDB 10.2 보다 이전 버전 사용시 주의
  - InnoDB 지정 필요
  - Default는 MyISAM

Default에서
는 처리 안됨

```
CREATE TABLE IF NOT EXISTS `reply` (
        `replyid` int(11) NOT NULL AUTO_INCREMENT,
        `reviewid` int(11) NOT NULL,
        `userid` int(11) NOT NULL,
        `memo` text NOT NULL,
        `time` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        PRIMARY KEY (`replyid`),
        FOREIGN KEY (`reviewid`) REFERENCES `review`(`reviewid`) ON DELETE CASCADE ON UPDATE CASCADE,
        FOREIGN KEY (`userid`) REFERENCES `user`(`userid`) ON DELETE CASCADE ON UPDATE CASCADE,
        INDEX (`reviewid`),
        INDEX (`time`)
) ENGINE=InnoDB default character set utf8mb4 collate utf8mb4_unicode_ci;
```

# Primary key 및 Foreign key 예제

- R1 ≠ R2인 경우
  - 교수 (<u>교수번호</u>, 교수이름, 학과번호, 직급)
  - 학과 (<u>학과번호</u>, 학과이름, 학과장교수번호, 학생수)
  - PK FK
  - 학생 (<u>학번</u>, 이름, 학년, 학과)
  - 과목 (<u>과목번호</u>, 과목이름, 학점, 학과, 담당교수)
  - 등록 (<u>학번, 과목번호</u>, 성적)
  - FK FK
- R1 = R2인 경우
  - 교수1 (<u>교수번호</u>, 교수이름, 학과번호, 학장교수번호)
  - PK FK

# 나머지 무결성

- 도메인 무결성
  - 속성 값은 정의된 도메인에 속해야 함
- 사용자 정의 무결성
  - 속성 값은 사용자가 정의한 제약 조건을 만족해야 함
    - 공급자 번호는 Snnnn 형태
    - 공급자 상태 값은 반드시 1~100 범위내
    - 부품 중량은 0보다 큼
- 무결성 보장 방법
  - 제약 조건 검사
    - WEIGHT INTEGER CHECK (WEIGHT > 0)
    - CREATE ASSERTION VALID_WEIGHT CHECK (WEIGHT > 0)
  - 트리거
    - CREATE TRIGGER

# Summary : Integrity Constraints

- (1) 개체 무결성 (entity integrity)
  - Primary key 값은 언제 어느 때고 null값을 가질 수 없다.

Garbage
(쓰레기)
개념과 비슷

Dangling
Reference
개념과 비슷

- (2) 참조 무결성 (referential integrity)
  - Foreign key 값은 참조되는 relation의 primary key 값이거나 null이다.

- 데이터베이스 상태(database state)가 항상 만족되어야 함
  - DBMS는 데이터베이스 상태의 변화에도 항상 무결성 제약을 만족시키도록 해야 함
  - 어느 한 시점에 데이터베이스에 저장된 데이터 값
  - Database instance
  - 데이터베이스 상태 변화: insert, delete, update 연산